

AUTONOMOUS AI BUG FIXING ASSISTANT

Poonawalla Fincorp

Siya Verma 26A126



Problem Statement

The Problem-

Software teams spend significant time on:

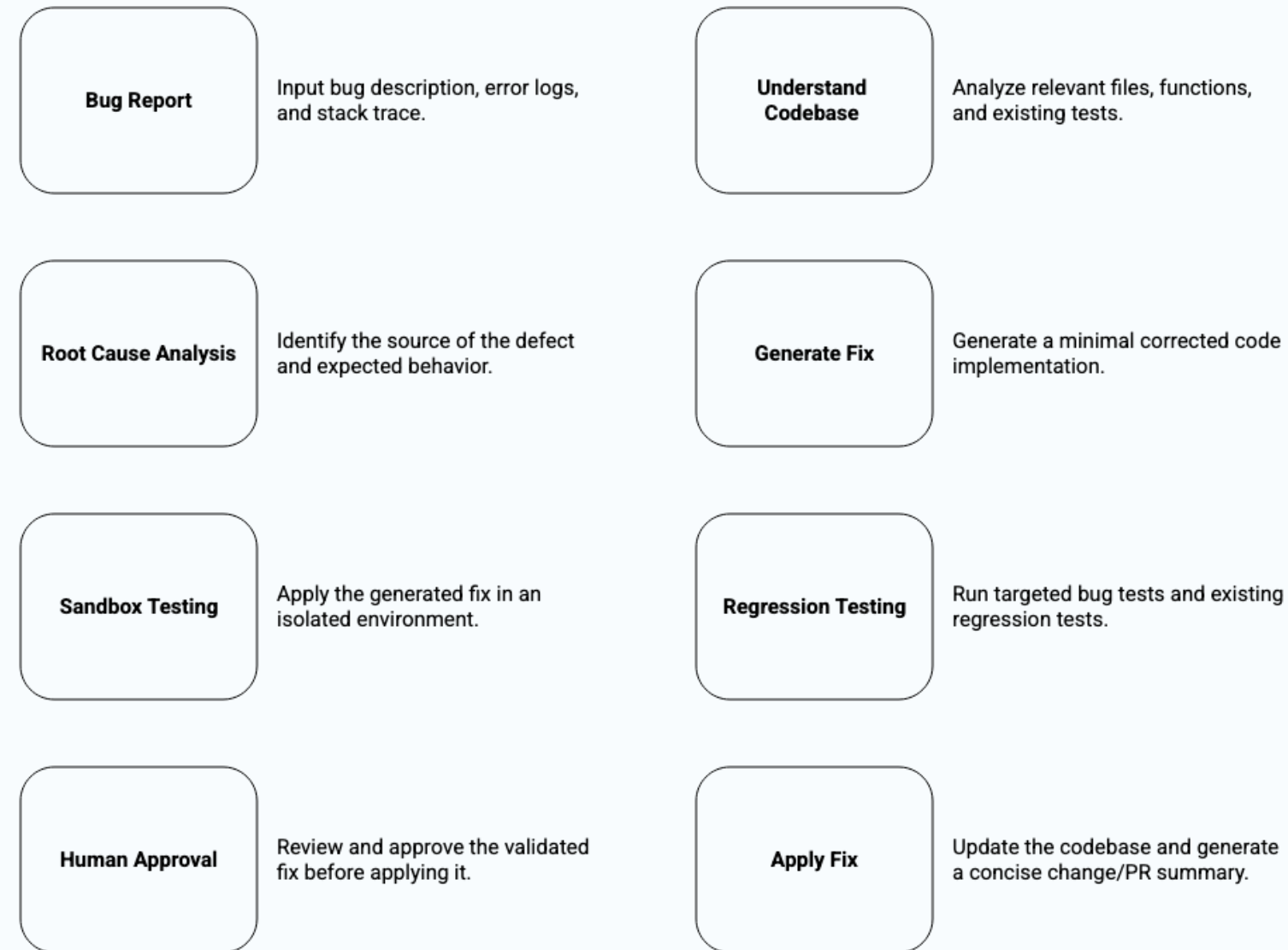
- Understanding bug reports
- Tracing errors through existing code
- Identifying root causes
- Implementing fixes
- Testing fixes
- Performing regression testing
- Documenting changes

Objective-

Automate the complete bug-resolution lifecycle while keeping validation and approval controlled.

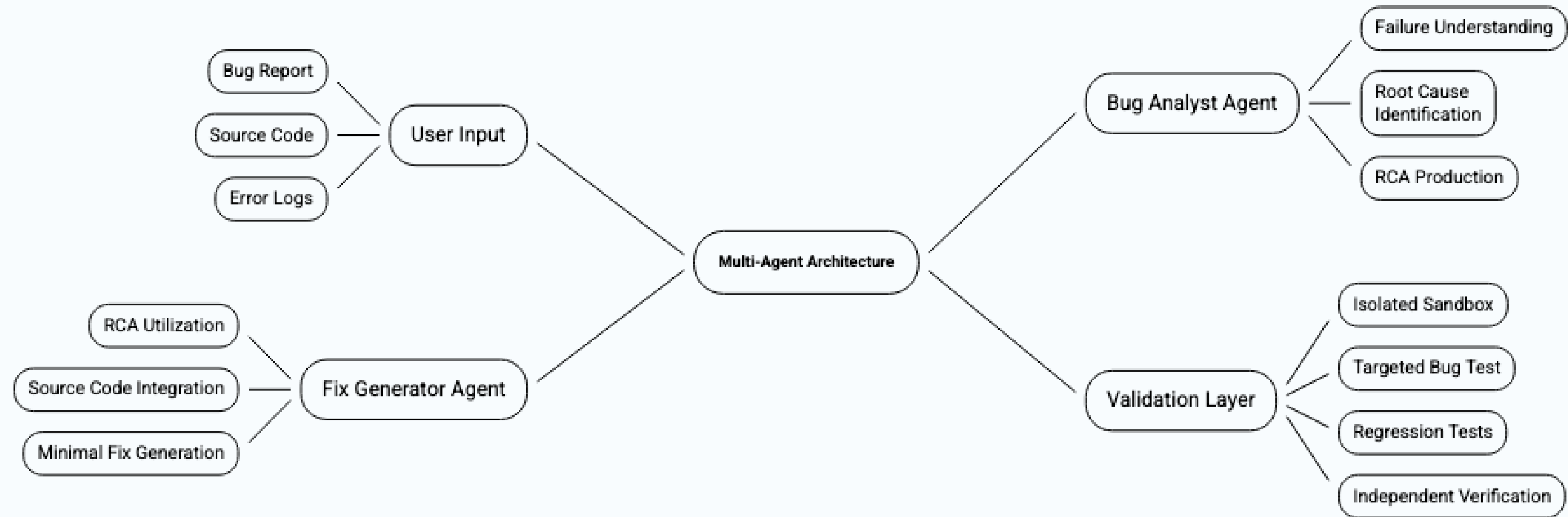


Proposed Solution

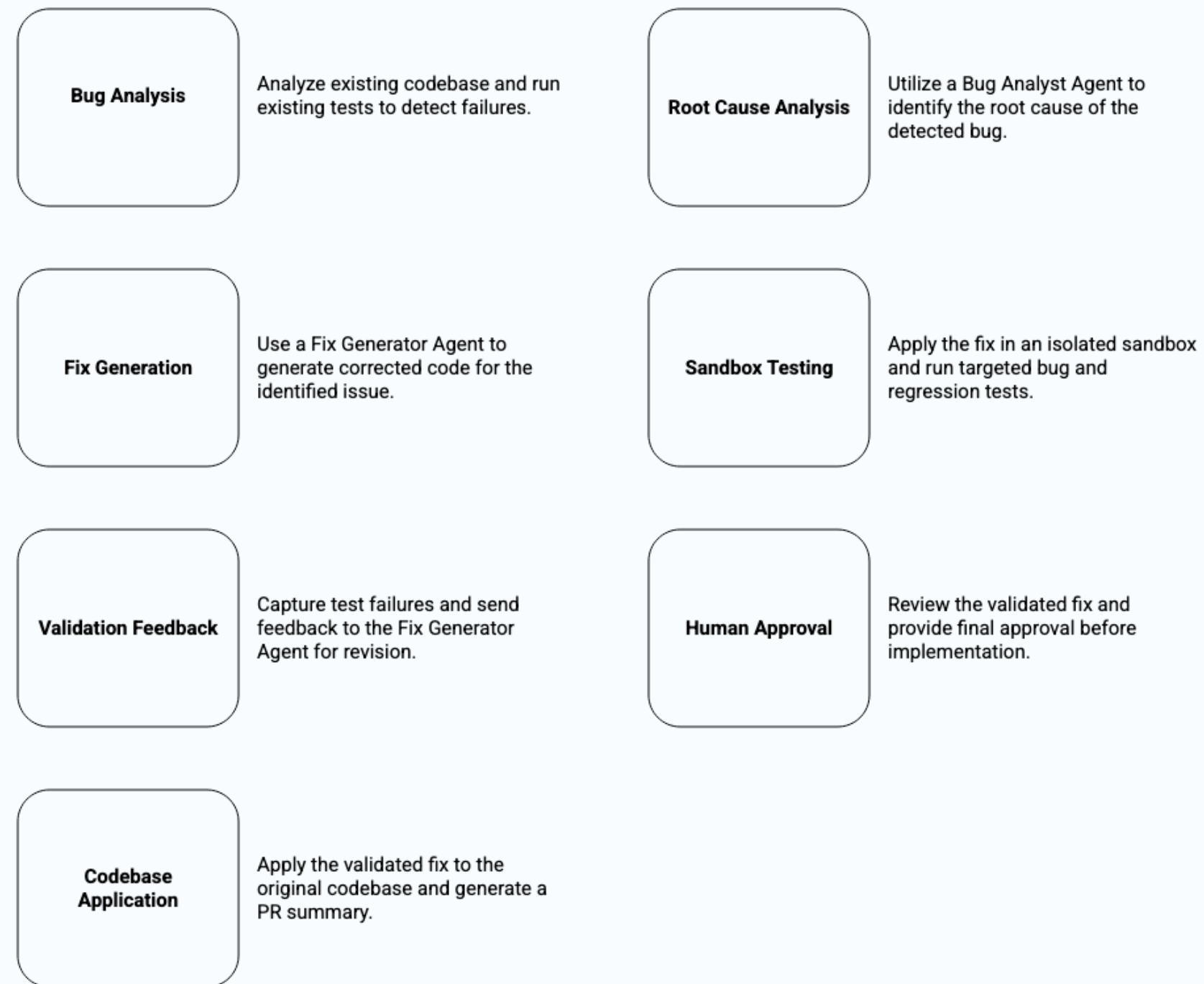


AI handles repetitive debugging work;
validation protects the codebase.

Multi-Agent Architecture



End-to-End Workflow



Demo: Bug Detection & RCA

Input

divide (10, 0)

Observed Error

ZeroDivisionError: division by zero

AI Output

Root Cause: Missing zero-denominator check

```
tests/test_calculator.py ...F [100%]

===== FAILURES =====
----- test_divide_by_zero -----

def test_divide_by_zero():
> assert divide(10, 0) is None
      ^^^^^^^^^^^^^^^^^

tests/test_calculator.py:12:
-----
a = 10, b = 0

def divide(a, b):
> return a / b
      ^^^^^
E      ZeroDivisionError: division by zero

app/calculator.py:10: ZeroDivisionError
===== short test summary info =====
FAILED tests/test_calculator.py::test_divide_by_zero - ZeroDivisionError: division by zero
===== 1 failed, 3 passed in 0.02s =====
```

Generated Fix & Validation

```
Initial tests failed. Proceeding with AI analysis.
```

```
[3] Running Bug Analyst Agent...
```

```
Direct use of automatic function calling (AFC) in Models.generate_content is not recommended. Instead, we recommend to use AFC in Chat.send_message. Similarly, direct use of AFC in Models.generate_content_stream is not recommended. Instead, we recommend to use AFC in Chat.send_message_stream.
```

```
=== ROOT CAUSE ANALYSIS ===
```

```
1. **Bug Summary**:
```

```
The `divide` function crashes with a `ZeroDivisionError` when the second argument (`b`) is zero, instead of handling the condition gracefully.
```

```
2. **Root Cause**:
```

```
The function attempts to perform a direct division (`a / b`) without checking if the divisor (`b`) is equal to zero.
```

```
3. **Faulty Code Location**:
```

```
- File: `app/calculator.py`
```

```
- Function: `divide(a, b)` (specifically the return statement `return a / b`)
```

```
4. **Expected Behavior**:
```

```
When the denominator is zero, the function should return `None` instead of crashing.
```

```
5. **Suggested Fix**:
```

```
Add a conditional check inside the `divide` function to return `None` if `b == 0` before executing the division.
```

```
=== SANDBOX VALIDATION ===
```

```
Sandbox: outputs/sandbox_run_20260929_174812
```

```
--- TARGETED BUG TEST ---
```

```
Passed: True
```

```
===== test session starts =====
```

```
platform darwin -- Python 3.14.6, pytest-9.1.1, pluggy-1.6.0
```

```
rootdir: /Users/siyaverma/ai-bug-fixing-assistant/outputs/sandbox_run_20260929_174812
```

```
plugins: anyio-4.15.1
```

```
collected 4 items / 3 deselected / 1 selected
```

```
tests/test_calculator.py .
```

```
[100%]
```

```
===== 1 passed, 3 deselected in 0.01s =====
```

```
--- REGRESSION TESTS ---
```

```
Passed: True
```

```
===== test session starts =====
```

```
platform darwin -- Python 3.14.6, pytest-9.1.1, pluggy-1.6.0
```

```
rootdir: /Users/siyaverma/ai-bug-fixing-assistant/outputs/sandbox_run_20260929_174812
```

```
plugins: anyio-4.15.1
```

```
collected 4 items / 1 deselected / 3 selected
```

```
tests/test_calculator.py ...
```

```
[100%]
```

```
===== 3 passed, 1 deselected in 0.01s =====
```

```
=====
```

```
FIX VALIDATED SUCCESSFULLY
```

```
=====
```

```
Successful validation on attempt 1.
```

Safety & Control

Controlled autonomy

- Generated code is not immediately applied
- Fix is first tested in an isolated sandbox
- Targeted test validates the reported bug
- Regression tests check existing functionality
- Human approval is required before modifying the original repository
- Failed validation can trigger another bounded fix attempt



Outcome & Future Scope

Current Prototype

- LLM-powered bug analysis
- AI-generated code fixes
- Multi-agent workflow
- Isolated sandbox validation
- Automated testing
- Regression testing
- Human approval
- Change/PR summary

Future Scope

- Larger repositories
- More programming languages
- Container-based sandbox isolation
- Git-based patch/PR integration
- Repository-level context retrieval
- More sophisticated agent orchestration



THANK YOU